

MVP Problem Statement

“TieBreaker” — Wishlist Comparison Agent

Part 5 build spec · Myntra Growth · for use in Cursor / any AI coding assistant

Read this whole document before writing code — it defines what to build, what data to use, and what is explicitly out of scope.

1. Context — Why This MVP Exists

Business goal: Increase the % of users who purchase at least one wishlisted item within 30 days of adding it.

Constraint: no monetary incentives (no discounts, coupons, cashback, or price-offs as the primary mechanism).

Validated root cause (AI discovery over 17,495 relevant records + a 36-respondent primary-research survey — see reports/problem_definition.md and reports/opportunity_register.md):

Users wishlist an item while still comparing it against alternatives, intending to resolve that comparison later. But the wishlist is a passive bookmark, not a decision-support workspace — it captures no reason for the save, has no mechanism to pull the user back before attention decays, and gives no in-app way to actually resolve the comparison. The result: either a competitor wins off-platform (42% of abandoned items) or the user simply forgets the item exists (32% of true non-converters — the single largest blocker).

This MVP directly targets that root cause. It does **not** target returns/exchange, delivery, or price — those were investigated and explicitly ruled out as the primary driver.

2. What to Build — One-liner

“TieBreaker” — a standalone AI-powered experience where a user builds a small wishlist from real Myntra product data, tags *why* they saved each item, gets nudged back before attention would naturally decay, and gets an AI agent's help actually resolving the comparison in-app.

3. Data to Use (already in the repo — do not fabricate product data)

Product catalog:

data/raw/huggingface/Gssmc__myntra_dataset/train.jsonl

15,000 real Myntra product listings (JSONL, one product per line). Relevant fields: title, brand, product_type, dominant_material, dominant_color, size_fit, product_details, complete_the_look, variant_price, ideal_for, is_in_stock, specifications. Field variant_compare_at_price also exists — **do not surface this or any discount/markdown framing anywhere in the UI or agent output; it violates the no-monetary-incentive constraint.**

Filter this file at build/load time to a coherent demo subset (e.g. a couple of product_type values such as Tops/Kurtas, in-stock only) so comparisons in the UI are apples-to-apples, not 15,000 random SKUs. **Do not hand-write mock products.**

LLM client:

Reuse src/analyze/llm.py, specifically chat_json(messages_body: dict) -> dict. It already handles OpenAI/Groq-compatible endpoints, retries, and JSON-content parsing from environment variables

(OPENAI_API_KEY, OPENAI_BASE_URL, LLM_MODEL). Do not hardcode API keys; do not build a second LLM client.

4. Tech Stack & Module Layout

Streamlit, same as the existing research console (src/dashboard/), so it deploys to Streamlit Community Cloud the same way — no new infra.

New, separate module — do not touch the discovery pipeline (src/ingest, src/process, src/analyze, src/synthesize, src/score, src/dashboard, src/ideate):

```
src/mvp/  
  __init__.py  
  __main__.py      # entrypoint: python -m src.mvp  
  catalog.py       # loads + filters the real product sample  
  state.py         # session-state wishlist model (reason tag, added_at, decided status)  
  agent.py         # builds the comparison prompt, calls chat_json, validates the response  
  app.py           # Streamlit UI / page flow
```

Root entrypoint for deployment: `mvp_app.py` at repo root (mirrors the existing `streamlit_app.py` pattern), pointing at `src.mvp.app:main`.

Secrets in environment variables only (`.env`, already gitignored). Do not commit keys.

5. User Flow — Functional Requirements

Step	Requirement
1. Browse & wishlist	Show the filtered product sample as cards (title, brand, price, one-line detail). User adds 4–6 items to a wishlist. No login — one browser session is one user (session-state, not a database, for v1).
2. Capture the reason	Required one-tap chip on each add: Comparing with something else / Waiting for an occasion / Not sure about fit / Just liked it . Only items tagged “Comparing” are eligible for grouping into a comparison set (manual selection of 2–3 items, or auto-matched by product_type).
3. Fast-forward control	A clearly-labeled “Fast-forward N days” control simulates elapsed time (stands in for push notifications / real elapsed time — must be visibly labeled as a simulation, not hidden as real). Past a threshold (e.g. 7+ simulated days), stale Comparing-tagged items surface a nudge card.
4. Resolve — the agent	“Help me decide” (from the nudge, or any time on a comparison set) calls the agent with the real attributes of the set’s items plus the user’s stated reason. Renders: what’s actually different, which item best matches the stated reason, and why.
5. Decide	Explicit per-item action: Buy this one / Keep comparing / Remove from wishlist . Log the action in-session — this is the outcome metric: a comparison that resolved instead of going cold. Show a simple resolved-vs-cold tally in the UI.

6. Agent Contract (agent.py)

Input to `chat_json`: a prompt built from the comparison-set items' real attributes (title, brand, material, size_fit, product_details, occasion/ideal_for) plus the user's stated reason for each item. **Never include price-vs-discount framing or any monetary nudge in the prompt or the allowed output.**

Required JSON output shape (validate before rendering; do not trust the LLM to always comply):

```
{
  "summary": "one or two sentences on what's actually different between the items",
  "items": [
    {
      "id": "<catalog id>",
      "fit_for_stated_reason": "why this item does or doesn't resolve the user's stated uncertainty",
      "recommended": true
    }
  ],
  "recommendation_rationale": "why the recommended item (if any) is the pick, tied to what the
                             user said they cared about"
}
```

If the LLM call fails or returns invalid JSON, fail visibly in the UI (e.g. “Couldn't reach the assistant — try again”) — do not silently fall back to a fabricated recommendation.

7. Explicit v1 Scope Boundaries

In scope	Out of scope for v1 — do not build
Real product data, real LLM reasoning	Real push notifications or email (fast-forward stands in)
Reason-capture and decide UI	User accounts / auth / persistence across sessions or devices
Fast-forward simulation (clearly labeled)	Cross-app / competitor-site scraping or comparison
Deployment to a public, clickable URL	Any discount, coupon, price-drop, or cashback messaging anywhere in the product
	Editing or overwriting the discovery pipeline's data (data/raw/, data/processed/, etc.) — this MVP only <i>reads</i> the raw catalog file

8. Definition of Done

- Deployed to a public URL (Streamlit Community Cloud, matching `streamlit_app.py`'s existing deployment pattern) that a reviewer can open and click through without setup.
- A user can complete the full loop end to end: wishlist real items → tag a reason → fast-forward → get nudged → invoke the agent → get a real (not canned) comparison → make an explicit decide action.
- No monetary incentive appears anywhere in copy, UI, or agent output.
- No fabricated product data — everything shown traces back to `data/raw/huggingface/Gssmc__mynta_dataset/train.jsonl`.
- Secrets only via environment variables; no keys committed.